

Phân tích và thiết kế thuật toán

Ghi chép bởi Nguyễn Hải Cường - 67CS1

Mục lục

Giới thiệu môn học	2
Một số khái niệm cơ bản	3
Đánh giá độ phức tạp và chứng minh tính đúng	5
1 Đệ quy	7
1.1 Khái niệm	7
1.2 Đặc trưng của các bài toán	7
1.3 Một số ví dụ	7
1.4 Độ phức tạp giải thuật đệ quy	9
1.5 Giải công thức truy hồi	9
2 Chia để trị	10
2.1 Giới thiệu	10
2.2 Bài toán tìm kiếm nhị phân	10
2.3 Bài toán min – max	11
2.4 Bài toán tìm cặp điểm gần nhất	12
3 Phương pháp thử sai	13
3.1 Phương pháp vét cạn	13
3.2 Phương pháp quay lui	13
4 Phương pháp nhánh cận	17
4.1 Tìm cấu hình khi biết giá trị của hàm mục tiêu	17
4.2 Tìm giá trị nhỏ nhất của hàm mục tiêu	17
4.3 Tìm giá trị lớn nhất của hàm mục tiêu	18
5 Quy hoạch động	19
5.1 VD2: Bài toán gom sỏi (Stones)	19
5.2 VD3: Biểu đồ tam giác	21
5.3 VD4: Tối ưu chi phí nhân dãy ma trận	22
5.4 VD5: Bài toán cái ba-lô 0-1	22
5.5 VD6. Bài toán nhân dãy ma trận với chi phí thấp nhất	23

Giới thiệu môn học

I. Mục tiêu:

Kiến thức:

- Trang bị kiến thức cơ bản về thiết kế và đánh giá thuật toán
- Cung cấp một số thuật toán cơ bản

Kỹ năng

- Hiểu được quá trình phân tích bài toán xây dựng thuật toán, đánh giá độ phức tạp của thuật toán
- Triển khai chương trình thể hiện các thuật toán

II. Nội dung

- Các vấn đề về lý thuyết thuật toán
- Các phương pháp thiết kế thuật toán
- Các thuật toán thông dụng

Một số khái niệm cơ bản

I. Bài toán (Problem)

VD1: Sắp xếp một dãy số

- Cho một dãy số $a = (a_1, a_2, \dots, a_n)$
- Cần xác định dãy $a' = (a'_1, a'_2, \dots, a'_n)$ là hoán vị của a sao cho $a'_1 \leq \dots \leq a'_n$

VD2: Tìm đường đi ngắn nhất từ u tới v của đồ thị $G = (V, E)$

- Cho đồ thị G , và hai đỉnh u, v
- Cần xác định đường đi $d = (u = v_1, v_2, \dots, v_b = v)$ có độ dài ngắn nhất

Bài toán trong tin học: lớp các bài toán cụ thể cùng loại

Bài toán được cấu tạo bởi hai thành phần cơ bản:

- **Thông tin vào (input):** dữ liệu ta có
- **Thông tin ra (output):** yếu tố cần xác định

Cho bài toán: cho input và output

Giải bài toán: từ input, dùng một số hữu hạn các bước thao tác để tìm được output theo yêu cầu của bài toán

II. Thuật toán (Algorithm)

Là một dãy hữu hạn các thao tác đơn giản được sắp xếp theo một trình tự xác định dùng để giải một bài toán

Đặc trưng của các thuật toán: tổng quát (giải được các bài toán cùng dạng), dừng, xác định, hiệu quả (không/thời gian)

Phương pháp mô tả thuật toán: liệt kê từng bước, sơ đồ khối, giả mã

VD: bài toán người du lịch: tìm chu trình đi qua tất cả các (5) đỉnh của đồ thị đúng một lần (với 4! cách đi)

Nghiên cứu thuật toán quan tâm đến những vấn đề sau:

- **Tính giải được:** lớp bài toán nào giải được bằng thuật toán lớp bài toán không giải được bằng thuật toán
- **Tối ưu hóa thuật toán:** tìm những thuật toán tốt hơn

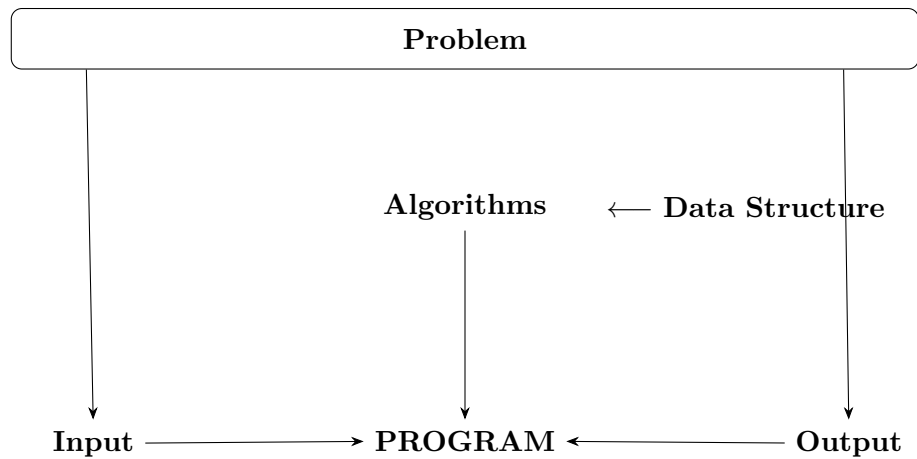
- **Triển khai thuật toán:** sử dụng các ngôn ngữ lập trình để thực hiện thuật toán trên máy tính

Hai lĩnh vực nghiên cứu chính: thiết kế và đánh giá thuật toán; triển khai thuật toán (lập trình)

III. Chương trình (Program)

Là sự thể hiện bằng một ngôn ngữ lập trình cụ thể một thuật toán đã cho được xác định trên một cấu trúc dữ liệu xác định

Sơ đồ giải bài toán trên máy tính:



Đánh giá độ phức tạp và chứng minh tính đúng

I. Đánh giá độ phức tạp

- **Phương pháp thực nghiệm:** Lập trình và thử trên các ví dụ
- **Phương pháp lý thuyết:** Tính toán thời gian, bộ nhớ,... cần thiết của mỗi thuật toán dựa theo độ lớn của input

Ưu điểm:

- Không phụ thuộc ngôn ngữ lập trình, loại máy tính
- Biết được tính hiệu quả của thuật toán đối với các dữ liệu có kích thước lớn

Đánh giá độ phức tạp: O (đọc là Ô lớn)

Cho $f(n)$: n là kích thước input

$f(n)$: không – thời gian

sử dụng khái niệm O

Ví dụ: Xét $T(n) = n^2 + n$

Với mọi $n \geq 1$: $T(n) \leq 2n^2 \Rightarrow T(n) = O(n^2)$

Nhận xét: Nếu $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = K$ (với $K < +\infty$) thì $f(n) = O(g(n))$. Chứng minh theo đ/n giới hạn của dãy số

Quy tắc đánh giá độ phức tạp thuật toán:

1. Quy tắc bỏ hằng số

Nếu $T(n) = O(c_1 \cdot f(n))$ thì $T(n) = O(f(n))$

CM: $T(n) = O(c_1 \cdot f(n)) \Rightarrow T(n) \leq c \cdot c_1 \cdot f(n)$ với mọi $n \geq n_0$

Chọn $c' = c \cdot c_1 \Rightarrow T(n) \leq c' \cdot f(n)$ với mọi $n \geq n_0 \Rightarrow T(n) = O(f(n))$

2. Quy tắc cộng

Nếu thuật toán P có hai thành phần liên tiếp P_1 với P_2 có độ phức tạp là $T_1(n) = O(f(n))$ và $T_2(n) = O(g(n))$ thì P có độ phức tạp là $T(n) = O(f(n) + g(n))$

CM: $T_1(n) \leq c_1 \cdot f(n)$ với mọi $n \geq n_1$; $T_2(n) \leq c_2 \cdot g(n)$ với mọi $n \geq n_2$

Chọn $c = \max(c_1, c_2)$; $n_0 = \max(n_1, n_2)$, ta có: $T(n) \leq c(f(n) + g(n))$ với $n \geq n_0$

3. **Quy tắc lấy max:**

$$T(n) = O(f(n) + g(n)) \text{ và } \lim \frac{f(n)}{g(n)} = 0 \Rightarrow T(n) = O(g(n))$$

4. **Quy tắc nhân**

Nếu thuật toán P được thực hiện bằng cách thực hiện n lần thuật toán P^*

Giả sử P^* có độ phức tạp là $T_{P^*}(n) = O(f(n))$ thì P có độ phức tạp là $T_P(n) = O(n \cdot f(n))$

CM: $T_{P^*}(n) \leq c \cdot f(n)$ với mọi $n \geq n_0$

$\Rightarrow T_P(n) = n \cdot T_{P^*}(n) \leq c \cdot n \cdot f(n)$ với mọi $n \geq n_0$

$\Rightarrow T_P(n) = O(n \cdot f(n))$

II. Chứng minh tính đúng của thuật toán

Chương 1

Đệ quy

1.1 Khái niệm

Giải thuật đệ quy: giải bài toán T được thực hiện bằng cách giải bài toán T' có dạng giống như T .

Yêu cầu để giải thuật đệ quy thỏa mãn tính dừng:

- T' phải đơn giản hơn T
- T' giải được (ko cần đệ quy) trong một số trường hợp nào đó (trường hợp suy biến)

VD:

Tính giá trị $S(n) = n!$

$$S(n) = n! \leftarrow n \cdot S(n-1) \leftarrow (n-1) \cdot S(n-2) \leftarrow \dots \leftarrow 2 \cdot S(1)$$

1.2 Đặc trưng của các bài toán

- Các bài toán phụ thuộc tham số
- Ứng với các giá trị đặc biệt nào đó của tham số thì bài toán có giải thuật để giải (trường hợp suy biến)
- Trong trường hợp tổng quát bài toán có thể quy về dạng tương tự với một bộ giá trị mới của tham số và sau một số hữu hạn lần thì có thể dẫn tới trường hợp suy biến.

1.3 Một số ví dụ

Xác định số thứ n của dãy số Fibonacci: 1, 1, 2, 3, 5, 8, 13...

Giải thuật tính $Fib(n)$:

- TH suy biến: $n = 1$ hoặc $n = 2$ thì $Fib(n) = 1$
- TH tổng quát: $Fib(n) = Fib(n-1) + Fib(n-2)$

$\Rightarrow$ Công thức tường minh xác định số Fibonacci thứ n ?

Bài toán tháp Hà Nội: chuyển n tầng tháp từ A sang B lấy C làm trung gian.

Điều kiện: chỉ có 3 cột để di chuyển, mỗi lần chuyển 1 tầng, trong quá trình chuyển mỗi tầng luôn được đặt trên tầng lớn hơn.

Giải thuật chuyển tháp *CHUYEN*(*A, B, C*):

- TH suy biến: $N = 1$ thì *Chuyen1Tang*(*A, B*)
- TH tổng quát ($N > 1$):

$$T(n) = \begin{cases} O(1) & \text{khi } n = 1 \\ 2T(n-1) + O(1) & \text{khi } n > 1 \end{cases}$$

```
CHUYEN(N-1, A, C, B);
Chuyen1Tang(A,B);
CHUYEN(N-1, C, B, A);
```

Trong đó *Chuyen1Tang*(*X, Y*) là thao tác chuyển một tầng tháp từ $X \rightarrow Y$

Lược đồ:

```
Recursive_Algorithm(p_n) =
    if (p_n = p_0) // suy bien
        <thuc hien giai thuat truong hop suy bien>;
    else //tong quat
        <lenh;>
        Recursive_Algorithm(p_{n-1})
End.
```

VD: Tính $n!$

```
S(n): int = //Ham de quy tinh gia tri
    n giai thua.
    if (n==1)
        S = 1;
        n>1
    else
        S = n * S(n-1)
End.
```

$$\begin{aligned} T(n) &= O(1) \text{ khi } n=1 \\ &= T(n-1) + O(1) \text{ khi } n>1 \\ T(n) &= T(n-1) + O(1) \\ &= T(n-2) + O(2) \\ &= T(1) + O(n-1) \\ &= O(1) + O(n-1) = O(n) \end{aligned}$$

Tính $Fib(n)$:

```
Fib(n):int =
    if (n==1|n==2)
        Fib = 1;
        0(1) khi n > 2
    else
        Fib = Fib(n-1) + Fib(n-2);
End.
```

$$\begin{aligned} T(n) &= O(1) \text{ khi } n = 1, 2 \\ &= T(n-1) + T(n-2) + O(1) \text{ khi } n > 2 \end{aligned}$$

1.4 Độ phức tạp giải thuật đệ quy

- $T(n)$: độ phức tạp của giải thuật đệ quy với bài toán kích thước n
- $O(1)$: độ phức tạp thuật toán trong trường hợp suy biến khi $n = n_0$
- $f(n)$: hàm biến đổi tham số n
- Nếu giải thuật được thực hiện bằng cách thực hiện a lần bài toán con với tham số $f(n)$ thì:

$$T(n) = a \cdot T(f(n)) + g(n)$$

($g(n)$ là độ phức tạp các thao tác ngoài lời gọi đệ quy)

⇒ Công thức truy hồi xác định độ phức tạp giải thuật đệ quy:

$$T(n) = \begin{cases} O(1) & \text{khi } n = n_0 \text{ (suy biến)} \\ a \cdot T(f(n)) + g(n) & \text{khi } n > n_0 \text{ (tổng quát)} \end{cases}$$

VD:

$$\begin{aligned} T(n) &= 2 \cdot T(n-1) + O(1) = 2[2 \cdot T(n-2) + O(1)] + O(1) \\ &= 2^2 \cdot T(n-2) + O(2) + O(1) \\ &= 2^2[2 \cdot T(n-3) + O(1)] + O(2) + O(1) \\ &= 2^3 \cdot T(n-3) + O(2^2) + O(2) + O(1) \\ &= 2^{n-1} \cdot T(1) + O(1 + 2 + \dots + 2^{n-2}) \\ &= O(2^{n-1}) + O(2^{n-1}) \\ &= O(2^n) \end{aligned}$$

1.5 Giải công thức truy hồi

Định lí chính: với n là lũy thừa của b , công thức truy hồi:

$$T(n) = \begin{cases} O(1) & \text{khi } n = 1 \\ a \cdot T(n/b) + O(n^d) & \text{khi } n > 1 \end{cases}$$

(với $a \geq 1, b > 1, d \geq 0$)

Có lời giải:

$$T(n) = \begin{cases} O(n^d) & \text{khi } a < b^d \\ O(n^d \cdot \log n) & \text{khi } a = b^d \\ O(n^{\log_b a}) & \text{khi } a > b^d \end{cases}$$

Chứng minh bằng sử dụng phép thế liên tiếp

Chương 2

Chia để trị

2.1 Giới thiệu

Ý tưởng giải thuật chia để trị:

Là phương pháp thiết kế thuật toán dựa trên 2 thao tác chính:

- **Chia (divide):** phân rã bài toán ban đầu thành các bài toán có kích thước nhỏ hơn và có cùng cách giải với bài toán ban đầu
- **Trị (conquer)**

Mô hình:

Xét bài toán P trên miền dữ liệu R

Gọi $D\&C(R)$ là thuật giải P miền dữ liệu R

Nếu R có thể phân rã thành n miền con ($R = R_1 \cup R_2 \cup \dots \cup R_k$)

Với R_0 là miền đủ nhỏ để $D\&C(R_0)$ có lời giải thì lược đồ giải thuật chia để trị

2.2 Bài toán tìm kiếm nhị phân

Bài toán: Tìm kiếm nhị phân trên mảng được sắp

Cho mảng n phần tử được sắp theo thứ tự (tăng) và một gt x bất kỳ

Kiểm tra xem x có trong dãy không? Nếu có, trả về vị trí của x

Ý tưởng: so sánh giá trị x với phần tử giữa của dãy tìm kiếm. Dựa vào giá trị này sẽ quyết định giới hạn tìm kiếm ở bước kế tiếp là nửa trước hay nửa sau dãy.

VD: 1 3 5 10 12 14 18 20

Duyệt mảng 1 lần $\Rightarrow$ độ phức tạp là $O(n)$

$$T(n) = \begin{cases} O(1) & \text{khi } n = 1 \\ T(n/2) + O(1) & \text{khi } n > 1 \end{cases}$$

$a = 1; b = 2; d = 0 \Rightarrow a = b^d$

$\Rightarrow T(n) = O(n^d \cdot \log n) = O(\log n) \Rightarrow$ Tốt hơn tìm kiếm tuần tự

Lược đồ:

```
BinarySearch (a, x, L, R): int =  
    if (L=R) return (x = a[L] ? L : -1)  
    else
```

```

        M = (L+R)/2;
        if (x = a[M]) return (M);
        else
            if (x < a[M]) BinarySearch (a, x, L, M-1)
            else BinarySearch(a, x, M+1, R)
        endif;
    endif
End.

```

2.3 Bài toán min – max

Bài toán: Cho mảng n phần tử, tìm giá trị lớn nhất và nhỏ nhất các phần tử trong dãy.

Phân tích: Phương pháp truyền thống: dùng hai vòng lặp độc lập tìm max, min.

- Độ phức tạp thuật toán $O(n)$
- Số phép so sánh $2(n-1)$

Phương pháp chia để trị:

- **Chia:** Chia dãy thành 2 phần
- **Trị:** Tìm giá trị max, min trong mỗi phần
- **Kết hợp:** So sánh giá trị min, max đã tìm được

Lược đồ:

```

MinMax(a, L, R): (int, int) =
    if (R-L <= 1) {
        return(min(aL, aR), max(aL, aR));}
    else {
        (min1, max1) = MinMax(a, L, (L+R)/2);
        (min2, max2) = MinMax(a, (L+R)/2+1, R);
        return(min(min1, min2), max(max1, max2));}
End.

```

$$T(n) = \begin{cases} O(1) & \text{khi } n \leq 2 \\ 2T(n/2) + O(1) & \text{khi } n > 2 \end{cases}$$

$$a = 2, b = 2; d = 0 \Rightarrow a > b^d$$

$$\Rightarrow T(n) = O(n^{\log_2 2}) = O(n)$$

Khi dùng chia để trị thì số phép so sánh là?

$f(n)$: Số phép so sánh

$$f(n) = \begin{cases} 1 & \text{khi } n = 2 \\ 2f(n/2) + 2 & \text{khi } n > 2 \end{cases}$$

Giả sử $n = 2^k$:

$$\begin{aligned}
f(n) &= f(2^k) = 2f(2^{k-1}) + 2 \\
&= 2(2f(2^{k-2}) + 2) + 2 \\
&= 2^{k-1} \cdot f(2) + 2 + \dots + 2^{k-1} \\
&= 2^{k-1} + 2 \cdot \frac{2^{k-1} - 1}{1} \\
&= \frac{1}{2} \cdot 2^k + 2^k - 2 \\
&= \frac{3n/2 - 2}{2n - 2} \rightarrow \frac{3}{4}
\end{aligned}$$

2.4 Bài toán tìm cặp điểm gần nhất

Bài toán: Tìm kiếm cặp điểm gần nhất

Cho một tập n điểm trong mặt phẳng, tìm cặp điểm gần nhau nhất (k/cách Euclid)

$P = \{(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)\}$

Tìm cặp điểm p_i, p_j sao cho $d_{ij} = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}$ nhỏ nhất

Nếu dùng phương pháp vét cạn: so sánh khoảng cách mọi cặp điểm $\Rightarrow$ độ phức tạp: $O(n^2)$

$\Rightarrow$ Độ phức tạp mong muốn là $O(n \cdot \log n)$

$$T(n) = \begin{cases} O(1) & \text{khi } n = 2 \\ 2T(n/2) + O(n^2) & \text{khi } n > 2 \end{cases}$$

$$a = b = d = 2 \Rightarrow a < b^d$$

$$\Rightarrow T(n) = O(n^d) = O(n^2)$$

Phân tích giải pháp:

- **Chia:** Phân chia tập n điểm P thành 2 tập con L_x, R_x (theo giá trị tăng dần của hoành độ)
- **Trị:** Tìm cặp điểm gần nhất trong mỗi tập L_x, R_x . Giả sử khoảng cách giữa hai điểm gần nhau nhất của L_x là δ_1 , của R_x là δ_2
- **Kết hợp:** So sánh và lấy ra cặp điểm gần hơn $\delta = \min(\delta_1, \delta_2)$
- **Chú ý:** Có thể có cặp (1 thuộc L_x , 1 thuộc R_x) có khoảng cách ngắn hơn
Cần xem xét thêm có các điểm có hoành độ thuộc khoảng $(x_m - \delta, x_m + \delta)$

Chương 3

Phương pháp thử sai

3.1 Phương pháp vét cạn

Ý tưởng: Duyệt tất cả các phương án có thể

Mô hình: Giả sử $D = \{(x_1, x_2, \dots, x_n)\}$ là không gian nghiệm của bài toán (tập khả năng)

Trong đó $x_i \in D_i$, với $D_i = \{d_{ij} \mid j = 1 \dots m_i\}$ là tập có m_i phần tử

Lược đồ và phương pháp:

```
VetCan(D) =  
  for (x1 = d11..d1m1)  
    for (x2 = d21..d2m2)  
      ...  
      for (xn = dn1..dnmn)  
        if (f(x1, x2, ..., xn))  
          <Chap nhan nghiem x = (x1, x2, ... xn)>;  
End.
```

VD: Tìm $x, y, z \in \mathbb{N}$ sao cho $x + y + z = 10$ (Bài toán: Tìm các bộ k số có tổng bằng n ($k, n \in \mathbb{N}$))

```
for x thuộc {0,... 10}  
  for y thuộc {0,... 10}  
    for z thuộc {0, ..., 10}  
      if x+y+z=10 then (x;y;z) la nghiem
```

{Sau khi xét hết các giá trị của z , thay đổi giá trị của y

Sau khi xét hết các giá trị của y , thay đổi giá trị của x = quay lui

$(x; y; z)$ là một cấu hình

Chỉ thích hợp khi độ dài cấu hình nhỏ, biết trước.

3.2 Phương pháp quay lui

Trong nhiều bài toán, việc tìm nghiệm có thể quy về tìm cấu hình $(x_1, x_2, \dots, x_n)$

- Độ dài của cấu hình có thể được xác định trước hoặc không
- Cấu hình phải thỏa mãn yêu cầu của bài toán

- Thành phần x_i được chọn từ tập hữu hạn A_i
- Yêu cầu của bài toán có thể là: tìm một nghiệm, tìm tất cả các nghiệm hoặc đếm số nghiệm

Các vấn đề quan trọng:

- Xác định được biểu diễn cấu hình
- Xác định tập S_i gồm những ứng viên cho x_i
- Xác định điều kiện để cấu hình là nghiệm

```

Main() {S={0;...;10};
      BackTrack(1;S)}

BackTrack(i; S)
{for v thuộc S
  {x(i) = v;
   max = max(S) - v;
   if i = 3 then
     {if x1+x2+x3 = 10
      then x1, x2, x3 la nghiệm}
    else {S = {0; max}
          BackTrack(i+1; S)}}
  }
}

```

Ý tưởng:

- Xây dựng các thành phần của cấu hình
- Trước khi xác định một thành phần của cấu hình, ta cần xây dựng tập S gồm những giá trị có thể có của thành phần đó
- Sau khi xác định thêm một thành phần của cấu hình, ta cần kiểm tra xem cấu hình hiện thời có phải là nghiệm không
- Ghi nhận nghiệm, sau đó có thể tìm thêm các nghiệm khác, hoặc là không.

VD: Bài toán sinh chỉnh hợp k của $A = \{1, ..., n\}$ phần tử.

Độ dài của cấu hình bằng k , gồm k số khác nhau có thứ tự

```

Main()
{S = A;
  Backtrack(1;S);}

Backtrack(i;S)
{for v thuộc S
  {x(i) = v;
   if i = k then {x1, x2, ..., xi} la nghiệm
   else {S = S\{v};
         Backtrack(i+1; S)}}
  }
}

```

Bài toán quân mã đi tuần

Cho bàn cờ vua 8×8 . Tìm các di chuyển của quân mã sao cho mỗi ô đi qua đúng MỘT lần.

Xét bàn cờ vua 5×5 :

1	14	7	12	?
8	?	?	3	18
15	2	13	6	11
?	9	4	17	?
?	16	?	10	5

$\Rightarrow$ Cấu hình cần xây dựng có độ dài là 64, gồm những nước đi liên tiếp của quân mã sao cho không có hai thành phần giống nhau.

Giải thuật:

```
Main()
{S = {(i;j) | i, j thuộc (1, 8)};
 D != rỗng // nhưng vị trí mã quân mã đã đi qua
 Backtrack(1;S;D)}

Backtrack(i;S;D)
{for v thuộc S
 {x(i) = v;
  if (i = 64) then x1, x2, ..., xi là nghiệm;
  else {D = D hợp {v};
        S = v.next() \ D; //nhưng ô quân mã có thể đi tới từ
        o v
        Backtrack(i+1;S;D)
      }
  }
}
```

$v(i;j) \Rightarrow v.next = \{(i \pm 1; j \pm 2); (i \pm 2; j \pm 1)\} \setminus \text{rìa ngoài}$

Bài toán 8 quân hậu

Tìm cách xếp 8 quân hậu vào bàn cờ 8×8 sao cho không có hai quân hậu chiếu nhau.

Xét bảng 5×5 :

H	?	?	?	?
?	?	?	H	?
?	H	?	?	?
?	?	?	?	H
?	?	H	?	?

Mỗi quân hậu phải nằm trên một hàng. Giả sử quân hậu ở hàng thứ i là quân hậu thứ i .

Cấu hình cần xác định là mảng $A[1..8]$ với $A[i]$ là chỉ số cột của quân hậu thứ i .

Ở VD trên, ta thấy một nghiệm của bài toán là $A = [1, 4, 2, 5, 3]$.

Chương 4

Phương pháp nhánh cận

Ý tưởng: giảm số TH phải xét khi xác định mỗi thành phần của cấu hình bằng cách sử dụng cận: cận trên, cận dưới.

4.1 Tìm cấu hình khi biết giá trị của hàm mục tiêu

VD: Cho $X = \{8; 6; 7; 5; 10; 3; 9\}$

Tìm các tập con của X biết tổng các phần tử của tập con bằng 12.

Range: $[3; 48]$

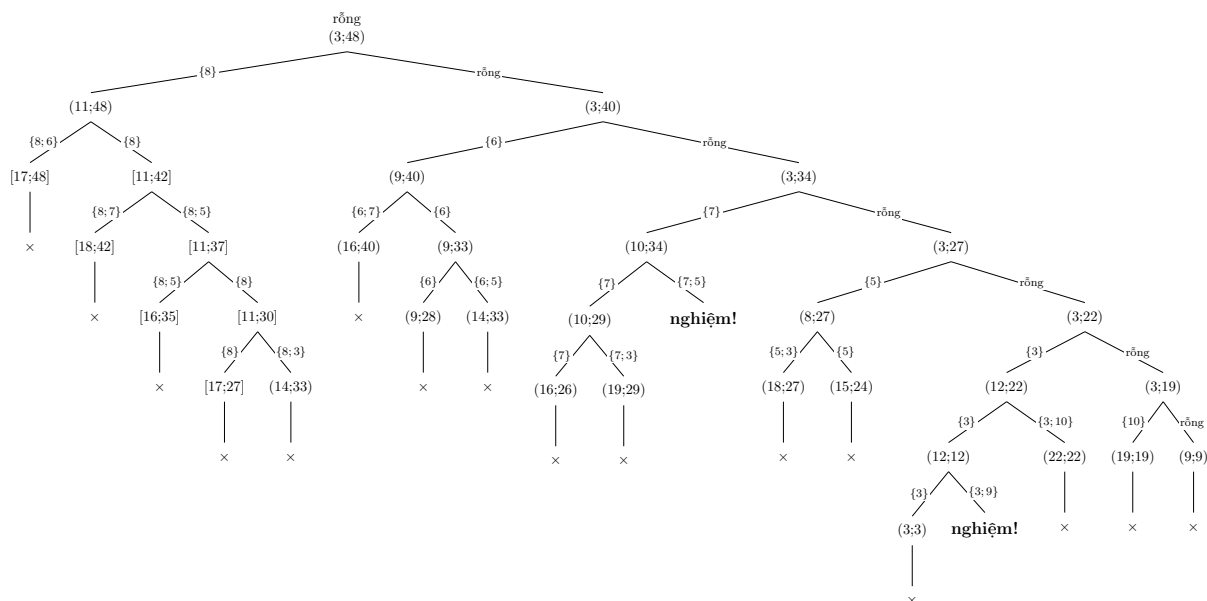
Giải: Cấu hình là dãy nhị phân $x_1, x_2, \dots, x_7$

Trước khi xác định thành phần tiếp theo của cấu hình, ta tìm đoạn $[CD; CT]$

Trong đó, CD là cận dưới của hàm mục tiêu và CT là cận trên của hàm mục tiêu.

Hàm mục tiêu là tổng các phần tử của tập con.

Nếu giá trị của hàm mục tiêu không thuộc $[CD; CT]$ thì cắt nhánh.



4.2 Tìm giá trị nhỏ nhất của hàm mục tiêu

VD: Bài toán phân công lao động

Cấu hình là dãy x_1, x_2, x_3, x_4 với $x_i \in \{A; B; C; D\}$

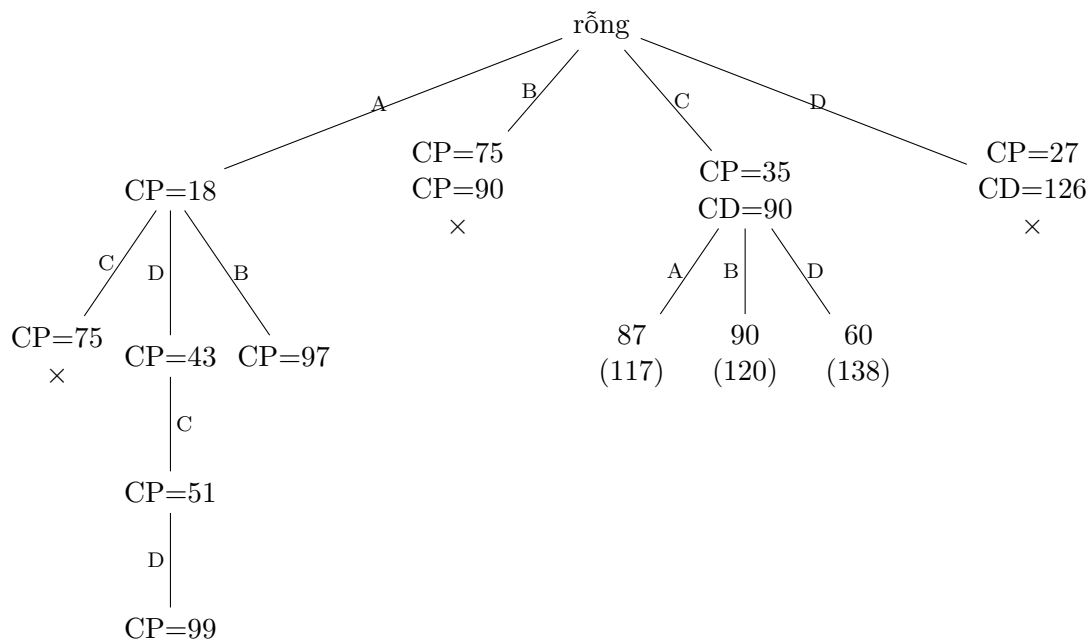
Nhân viên	Công việc I	II	III	IV
A	18	52	64	39
B	75	55	19	48
C	35	57	8	65
D	27	25	14	16

Khởi tạo chi phí tốt nhất hiện thời $\min = +\infty$

Ngay sau khi tìm được 1 lời giải, ta cập nhật $\min$

Trước khi xác định 1 thành phần của cấu hình, ta tìm cận dưới của giá trị hàm mục tiêu.

Nếu $CD > \min$ thì cắt nhánh. Ngược lại thì đi tiếp.



Vậy cấu hình tốt nhất là $ABCD$, với chi phí tối ưu là 97.

4.3 Tìm giá trị lớn nhất của hàm mục tiêu

Thay $\min$ bằng $\max$

Thay cận dưới bằng cận trên

Nếu nhỏ hơn $\max$ thì cắt nhánh, ngược lại thì đi tiếp.

VD: Bài toán cái ba-lô không giới hạn số lượng các đồ vật

Chương 5

Quy hoạch động

(mã: T: T; |: Y; t: t; y: y)

VD1: Sinh số Fibonacci thứ n .

Đệ quy: phức tạp.

Giải pháp: Dùng mảng để lưu kết quả.

F là mảng gồm nhiều phần tử

Khởi tạo $F[1] = F[2] = 1$

Duyệt mảng 1 lần để tính các phần tử. Còn lại: $F[k] = F[k-2] + F[k-1]$ (mọi $k \in [3, \infty)$)

$\Rightarrow$ Độ phức tạp là $O(n)$

1	1	2	3	5	8	...
---	---	---	---	---	---	-----

Ý tưởng:

- Xác định được cấu trúc dữ liệu phù hợp để lưu kết quả (mảng 1 chiều hoặc nhiều chiều)
- Xác định những giá trị biên
- Xác định công thức truy hồi để tính các giá trị còn lại
- Truy vết để tìm nghiệm

5.1 VD2: Bài toán gom sỏi (Stones)

Gom 1 đồng sỏi bằng cách lần lượt gom 2 đồng liền nhau làm một. Chi phí gom hai đồng bằng tổng số viên sỏi của hai đồng đó.

Tìm ra cách gom sao cho chi phí nhỏ nhất.

1	3	2	4	7	3
---	---	---	---	---	---

$$\begin{array}{cccccc}
1 & 3 & 2 & 4 & 7 & 3 \\
\hline
4 & & 2 & 4 & 7 & 3 & CP = 4 \\
\hline
6 & & & 4 & 7 & 3 & CP = 10 \\
\hline
10 & & & 7 & 3 & & CP = 20 \\
\hline
17 & & & & 3 & & CP = 37 \\
\hline
20 & & & & & & CP = 57
\end{array}$$

Giả sử có n đồng sỏi. Nếu vét cạn thì độ phức tạp là $O((n-1)!)$

Quy hoạch động:

Gọi $C[i; j]$ là chi phí nhỏ nhất để gom các đồng sỏi từ i đến j ($i \leq j \leq n$). Lưu kết quả vào mảng 2 chiều

Sử dụng VD từ dãy số trên: $\{1, 3, 2, 4, 7, 3\}$

Bảng $C[i; j]$:

$i \backslash j$	1	2	3	4	5	6
1	0	4	10	20	37	50
2	X	0	5	14	30	43
3	X	X	0	6	19	32
4	X	X	X	0	11	24
5	X	X	X	X	0	10
6	X	X	X	X	X	0

Giá trị biên: $C[i; i] = 0$ (mọi $i \in [1, \infty)$)

Công thức truy hồi: $C[i; j] = \min_{i \leq k < j} \{C[i; k] + C[k + 1; j]\} + T(i, j)$

$$(\text{VD: } C[2; 5] = C[2; 4] + C[5; 5] + T(2; 5) = 14 + 0 + 16 = 30)$$

Trong đó $T(i; j)$ là tổng số viên sỏi từ đồng i đến đồng j .

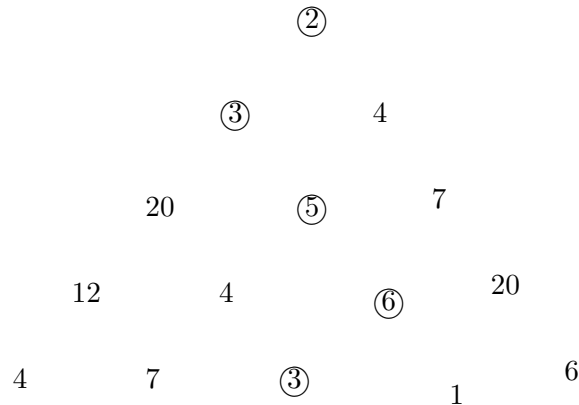
Bảng $T(i; j)$, với $1 \leq i \leq j \leq n$:

$i \backslash j$	1	2	3	4	5	6
1	1	4	6	10	17	20
2	X	3	5	9	16	19
3	X	X	2	6	13	16
4	X	X	X	4	11	14
5	X	X	X	X	7	10
6	X	X	X	X	X	3

$$(\text{VD: } T(2; 5) = 3 + 2 + 4 + 7 = 16)$$

5.2 VD3: Biểu đồ tam giác

Cho biểu đồ:



Tìm đường đi từ hàng 1 tới hàng n sao cho tổng các số trên đường đi là lớn nhất. Mỗi lần đi từ hàng i tới hàng $i + 1$, ta có thể đi chéo sang trái hoặc sang phải.

Nếu đi như ở trên () thì giá trị $= 2 + 3 + 5 + 6 + 3 = 19$

Vết cạn: Độ phức tạp là $O(2^n)$

Quy hoạch động:

Gọi $C[i; j]$ là giá trị tối ưu khi đi tới phần tử thứ j nằm trên hàng i ($n \geq i \geq j \geq 1$)

Bảng $C[i; j]$:

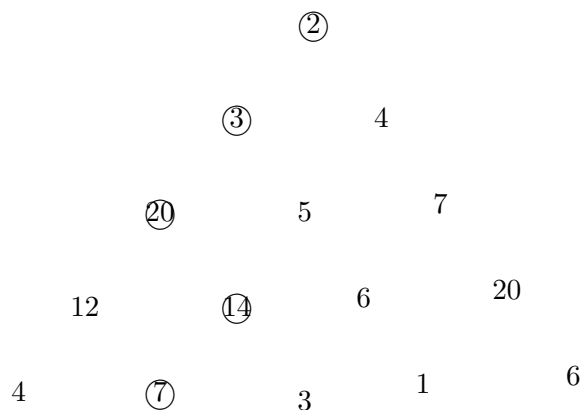
$i \backslash j$	1	2	3	4	5
1	2	X	X	X	X
2	5	6	X	X	X
3	25	11	13	X	X
4	37	33	19	33	X
5	41	46	42	34	39

$$C[i; 1] = A[1; 1] + A[2; 1] + \dots + A[i; 1]$$

$$C[i; i] = A[1; 1] + A[2; 2] + \dots + A[i; i]$$

$$C[i; j] = \max\{C[i-1; j-1], C[i-1; j]\} + A[i, j] \quad (1 < j < i)$$

Đường đi tối ưu:



Độ phức tạp: $O(n^2)$

5.3 VD4: Tối ưu chi phí nhân dãy ma trận

(gần giống với gom sỏi)

Chi phí nhân hai ma trận là số phép nhân hai số thực.

$$A \cdot B = C$$

$$(m \times n) \quad (n \times p) \quad (m \times p)$$

$$c_{ij} = \sum_{k=1}^n a_{ik} \cdot b_{kj} \Rightarrow \text{Chi phí nhân } A \text{ với } B \text{ là } mnp.$$

Với 3 ma trận:

$$A \cdot B \cdot C$$

$$(2 \times 3) \quad (3 \times 4) \quad (4 \times 6)$$

$$\text{TH1: } (A \cdot B) \cdot C$$

$$(2 \times 4) \cdot (4 \times 6) = 2 \cdot 3 \cdot 4 + 2 \cdot 4 \cdot 6 = 24 + 48 = 72$$

$$\text{TH2: } A \cdot (B \cdot C)$$

$$(2 \times 3) \cdot (3 \times 6) = 3 \cdot 4 \cdot 6 + 2 \cdot 3 \cdot 6 = 72 + 36 = 108$$

Với n ma trận: Vết cạn: Nhân dãy n ma trận: Độ phức tạp là $O((n-1)!)$.

5.4 VD5: Bài toán cái ba-lô 0-1

Tìm cách xếp các đồ vật vào ba-lô sao cho giá trị của các đồ vật là lớn nhất. DK: mỗi đồ vật có số lượng là 1.

Cho ba-lô có trọng lượng $W = 6$ và trọng lượng, giá trị các đồ vật như sau:

Đồ vật	Ký hiệu	Trọng lượng $W[i]$	Giá trị $P[i]$
1	A	3	25
2	B	2	20
3	C	1	15
4	D	4	40
5	E	5	50

$DP[i, j]$ là giá trị lớn nhất khi giải bài toán với i đồ vật đầu tiên, ba-lô có trọng lượng là j .

Ta cần xác định $DP[5, 6]$.

Giá trị biên: $DP[0, j] = DP[i, 0] = 0$ (với mọi $i \in [0, 6]; j \in [0, 5]$)

Bảng $DP[i, j]$:

$i \backslash j$	0	1	2	3	4	5	6
0	0	0	0	0	0	0	0
1	0	0	0	25	25	25	25
2	0	0	20	25	25	45	45
3	0	15	20	35	40	45	60
4	0	15	20	35	40	55	60
5	0	15	20	35	40	55	65

$i, j > 1 \Rightarrow DP[i, j] = ?$:

$$DP[i, j] = \begin{cases} DP[i-1, j] & \text{nếu } j < w[i] \\ \max\{DP[i-1, j]; DP[i-1, j-w[i]] + P[i]\} & \text{nếu } j \geq w[i] \end{cases}$$

Chi tiết tính toán:

$DP[1, 1] = DP[0, 1] = 0$ (vì $1 < w[1]$). Tương tự, $DP[1, 2]$.

$$DP[1, 3] = \max\{DP[0, 3]; DP[0, 0] + 25\} = 25$$

$$DP[2, 1] = DP[1, 1] = 0 \quad * \quad DP[3, 1] = 15$$

$$DP[2, 2] = \max\{DP[1, 2]; DP[1, 0] + 20\} = 20 \quad * \quad DP[3, 2] = \max\{DP[2, 2]; DP[2, 1] + 15\} = 20$$

$$DP[2, 3] = \max\{DP[1, 3]; DP[1, 1] + 20\} = 25 \quad * \quad DP[3, 3] = \max\{DP[2, 3]; DP[2, 2] + 15\} = 35$$

$$DP[2, 4] = \max\{DP[1, 4]; DP[1, 2] + 20\} = 25 \quad * \quad DP[3, 4] = \max\{DP[2, 4]; DP[2, 3] + 15\} = 40$$

$$DP[2, 5] = \max\{DP[1, 5]; DP[1, 3] + 20\} = 45 \quad * \quad DP[3, 5] = \max\{DP[2, 5]; DP[2, 4] + 15\} = 45$$

$$DP[2, 6] = \max\{DP[1, 6]; DP[1, 4] + 20\} = 45 \quad * \quad DP[3, 6] = \max\{DP[2, 6]; DP[2, 5] + 15\} = 60$$

$$DP[4, 1] = DP[3, 1] = 15 \quad * \quad DP[5, 1] = DP[4, 1] = 15$$

$$DP[4, 2] = DP[3, 2] = 20 \quad * \quad DP[5, 2] = DP[4, 2] = 20$$

$$DP[4, 3] = DP[3, 3] = 35 \quad * \quad DP[5, 3] = DP[4, 3] = 35$$

$$DP[4, 4] = \max\{DP[3, 4]; DP[3, 0] + 40\} = 40 \quad * \quad DP[5, 4] = DP[4, 4] = 40$$

$$DP[4, 5] = \max\{DP[3, 5]; DP[3, 1] + 40\} = 55 \quad * \quad DP[5, 5] = \max\{DP[4, 5]; DP[4, 0] + 50\} = 55$$

$$DP[4, 6] = \max\{DP[3, 6]; DP[3, 2] + 40\} = 60 \quad * \quad DP[5, 6] = \max\{DP[4, 6]; DP[4, 1] + 50\} = 65$$

Vậy 65 là giá trị lớn nhất khi lấy vật C và vật E.

5.5 VD6. Bài toán nhân dãy ma trận với chi phí thấp nhất

A có 2×3 B có 3×4 C có 2×2 D có 2×6

$$R = [2, 3, 4, 2]; C = [3, 4, 2, 6]$$

$DP[i, j]$ là chi phí thấp nhất để nhân các ma trận thứ i tới ma trận thứ j ($1 \leq i \leq j \leq 4$).

$R[i] \times C[k] \times C[j]$: nhân các ma trận từ i tới k được ma trận cỡ $R[i] \times C[k]$

$k + 1$ tới j : $R[k + 1] \times C[j]$

Bảng $DP[i, j]$:

$i \setminus j$	1	2	3	4
1	0	24	36	60
2	X	0	24	60
3	X	X	0	48
4	X	X	X	0

Giá trị biên $DP[i, i] = 0$ (với mọi $i \in [1, 4]$)

Công thức truy hồi cho $i < j$:

$$DP[i, j] = \min_{i \leq k < j} \{DP[i, k] + DP[k + 1, j] + R[i] \times C[k] \times C[j]\}$$

Chi tiết tính toán:

$$DP[1, 2] = DP[1, 1] + DP[2, 2] + R[1] \times C[1] \times C[2] = 24$$

$$DP[2, 3] = DP[2, 2] + DP[3, 3] + R[2] \times C[2] \times C[3] = 24$$

$$DP[3, 4] = DP[3, 3] + DP[4, 4] + R[3] \times C[3] \times C[4] = 48$$

$$\begin{aligned} DP[1, 3] &= \min\{DP[1, 1] + DP[2, 3] + R[1] \times C[1] \times C[3]; DP[1, 2] + DP[3, 3] + R[1] \times \\ &C[2] \times C[3]\} \\ &= \min\{0 + 24 + 12; 24 + 0 + 16\} = 36 \end{aligned}$$

$$\begin{aligned} DP[2, 4] &= \min\{DP[2, 2] + DP[3, 4] + R[2] \times C[2] \times C[4]; DP[2, 3] + DP[4, 4] + R[2] \times \\ &C[3] \times C[4]\} \\ &= \min\{0 + 48 + 72; 24 + 0 + 36\} = 60 \end{aligned}$$

$$\begin{aligned} DP[1, 4] &= \min\{DP[1, 1] + DP[2, 4] + R[1] \times C[1] \times C[4]; \\ &DP[1, 2] + DP[3, 4] + R[1] \times C[2] \times C[4]; \\ &DP[1, 3] + DP[4, 4] + R[1] \times C[3] \times C[4]\} \\ &= \min\{0 + 60 + 36; 24 + 48 + 48; 36 + 0 + 24\} = 60 \end{aligned}$$

Vậy chi phí thấp nhất là 60 khi nhân các ma trận: $ABCD = (ABC)D = [A(BC)]D$